

Tier-2 SOC Triage as an RL Task

Najmul Hasan

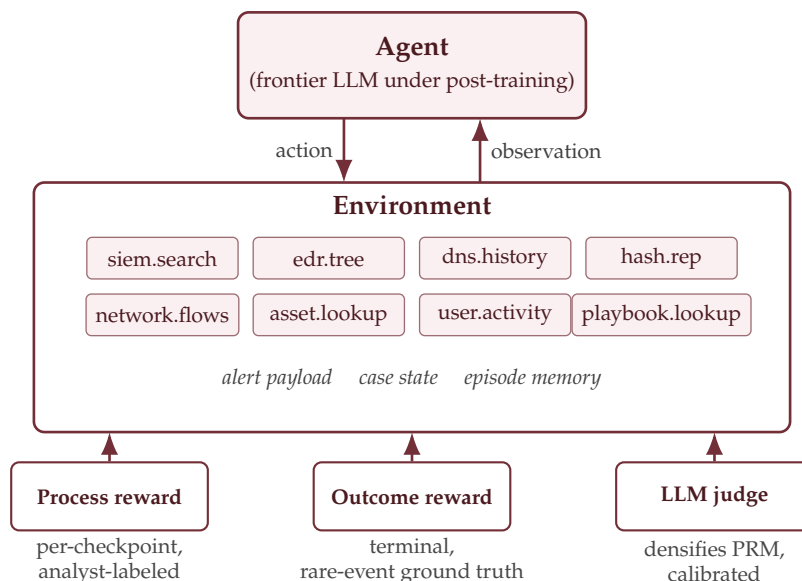


Figure 1: The RL loop for Tier-2 SOC triage. The agent emits actions (tool calls, case annotations, analyst messages, escalation or closure decisions). The environment is a structured world of eight tools plus alert payload, case state, and episode memory. Three reward streams compose: dense process rewards labeled by senior analysts at each checkpoint, sparse outcome rewards from the rare-event tail, and an LLM judge calibrated against the human-labeled set. Tool names in the figure are abbreviated; the body of the document uses their full names (edr.process_tree, hash.reputation).

Tier-2 SOC triage is the cleanest unverifiable long-horizon RL task I can design from first principles. Essentially every managed-security firm needs it, no frontier lab has publicly trained against it, and the workflow is long-horizon, multi-tool, multi-stakeholder, regulated, and rare-event reward, exactly the combination that exposes what current post-training pipelines cannot capture. This document specifies the environment, the harness, the reward model, and the quality control suite I would run before declaring it commercially viable.

Why this task

I worked these trajectories on the UNC Pembroke SOC floor from July 2023 to May 2026, building Cisco XDR automations and Python triage enrichment. The design choices below are downstream of which alerts actually waste a Tier-2's afternoon, not abstract guesses.

A Tier-2 trajectory is 45 to 90 minutes wall-clock, 10 to 25 tool calls, and three stakeholders (the analyst, the incident-response lead or operations manager who approves escalation, and the customer's security owner who receives the case). It carries no scalar ground truth at the trajectory level, and ground-truth incidents fire only in the long tail. The rest of this document operationalizes that into an RL task: environment, harness, reward model, QC suite.

Economic value is direct. Global managed security services revenue is around \$43B in 2026 (Mordor Intelligence), with a human-labor share that dominates delivery cost. Bottom-up: a typical Tier-2 escalation eats 30 to 90 minutes of analyst time at fully-loaded cost, and a mid-sized managed-security customer generates thousands of raw alerts daily that filter down to tens-to-hundreds of Tier-2 escalations. Even a 30% time reduction on the escalation queue lands as a five-figure monthly saving per customer. The capability this task targets, multi-turn investigative reasoning under tool constraints with rare-event reward, also transfers to agentic

coding sessions, deep research agents, and customer operations. The same point you make in "Kings and Electors" applies: labs ask for verifiable domains first because the reward signal is cheap, so unverifiable long-horizon ops is the fragment that gets left alone. Frontier labs are spending heavily on agentic post-training (Terminal-Bench 2.0, SWE-bench Pro, τ -bench, OSWorld are the public proxies), but none of those benchmarks live behind a customer-contract perimeter. The defensibility I care about is not the env spec (a lab can reimplement it in a quarter) but the captured trajectories themselves: a lab cannot replicate what came out of a firm whose customer-consent paperwork they did not negotiate.

The task is also clean to share with Anthropic. The data is procured under the consent and redaction structure I described in the neglected-work writeup, the reward model is grounded in a firm's own playbook rather than a contrived rubric, and the trajectories are typed by tool-call and decision rather than by free-text labels, which makes the bundles drop directly into HuggingFace chat templates.

Environment

The environment is a state machine with a fixed tool catalog and an episode budget. State, observations, actions, and tool schemas are Pydantic-typed throughout, so trajectories serialize cleanly into HuggingFace chat templates and tool calls validate at the boundary.

Observation space. At each step the agent receives a serialized observation containing the active alert payload (source, severity, indicators, timestamps, affected asset), the wall-clock timestamp, the available tool catalog with schemas, the case state (status, current severity, notes), and the episode memory. Memory uses incremental summarization (a rolling case-state summary maintained by the agent) when raw trajectories exceed the context budget; raw tool outputs over a size threshold are stored by reference and re-fetched on demand.

Action space. Discrete, typed actions in five families:

- `tool.query(name, args)`: invokes one of the catalog tools, returns a structured observation
- `case.update(field, value)`: annotates the case (severity, notes, indicators)
- `analyst.message(content)`: pushes a message to the human in the loop
- `decision.escalate(target, reason)`: terminal action, hands off to Tier-3 or incident response
- `decision.close(disposition, reason)`: terminal action, closes the case

Tool catalog. Eight tools cover the canonical pivot graph: `siem.search`, `edr.process_tree`, `dns.history`, `hash.reputation`, `network.flows`, `asset.lookup`, `user.activity`, `playbook.lookup`. Each has a Pydantic input schema and returns a structured output. In the pilot at a partner firm these tools hit real backends (SIEM, EDR, reputation services) through their existing APIs. In offline training and evaluation they hit deterministic mocks built from the captured trajectories described in the neglected-work writeup.

Episode budget and termination. Episodes terminate on `decision.escalate`, `decision.close`, or after 30 steps (hard ceiling). Step-count ceiling is enforced so that infinite-loop policies fail the QC. Median trained-agent episode length is targeted at 12 to 18 steps based on the captured human trajectories.

Compound alerts. With low probability a second alert can fire mid-episode on the same asset, simulating the long-tail of campaign activity. This stresses the agent's ability to update the case rather than process alerts in isolation. The probability is configured per training stage so that early SFT does not see compound events and later RL does.

Sample trajectory (representative, 7 steps)

```
t=0 ALERT umbrella, medium, dns_resolution, asset=hr-laptop-04, indicator=pszx9k2p.top
t=1 tool dns.history(host=hr-laptop-04, window=24h)
-> 1 resolution to indicator at 14:23, no other suspicious resolutions
t=2 tool edr.process_tree(host=hr-laptop-04, time=14:23)
-> WINWORD.EXE -> cmd.exe -> powershell.exe -> nslookup.exe
t=3 case update(severity, high)
t=4 tool hash.reputation(file=attachment.docm) -> 4/72 generic detections
t=5 tool network.flows(host=hr-laptop-04, 14:00-15:00)
-> TLS to known-bad C2 range at 14:24
t=6 decision escalate(target=tier3_ir, reason="C2 callback") [TERMINAL]
```

Figure 2: One trajectory in the environment (shorter than median to keep the figure compact). Process-reward checkpoints fire at steps 1, 2, 3, 4, 5, 6 (every tool call, every case update, and the terminal decision). Outcome reward fires after post-hoc breach review confirms the incident.

Harness

I would build on Verifiers (Prime Intellect’s open-source RL environment framework) rather than roll my own loop or fork a research stack like SkyRL. Verifiers gives me a small, trainer-agnostic surface (Env, SingleTurnEnv, MultiTurnEnv abstractions built on datasets, rubrics, and rollout harnesses), plus a native RLTrainer with first-class prime-rl integration and HuggingFace TRL compatibility. That matches the GRPO post-training stack I already shipped against in DPBench (GRPO on Qwen3-8B and Qwen3-14B with QLoRA), so the first pilot does not need me to debug two systems at once. The architecture also maps cleanly onto HUD’s production-software-as-RL-environment pattern through MCP, and since HUD is in your "New Age Commodities" map I keep the option to ship into their marketplace later without re-architecting.

The custom layer below Verifiers is small and well-scoped:

- `env.py`: episode lifecycle (reset, step, observation serialization, termination)
- `tools.py`: Pydantic schemas + async handlers for the eight tools, dispatching to real or mocked backends
- `reward.py`: composition of process, outcome, and judge streams
- `rollout.py`: a baseline runner against Claude Opus 4.7, GPT-5.5, Gemini 3.1 Pro, plus open-weights baselines (Qwen3-14B, GLM 5.1)

I would expect the full env package to land around one thousand lines of Python including tests. Small enough that one person ships and maintains it.

Reward model

Three streams compose into a single training signal.

Process reward $r_p(t)$. Scored at each tool-call, case-update, or terminal-decision checkpoint by senior analysts inside the partner firm. The rubric is the firm’s own playbook, which makes it economically meaningful rather than contrived. Scale is 3-point (incorrect / acceptable / preferred). Two reviewers per checkpoint, quadratic-weighted κ audited monthly with a 0.7 floor (substantial agreement on the ordinal scale, appropriate for safety-relevant labels).

Outcome reward r_o . Terminal, only on episodes that reach a decision.* action. Backfilled from post-hoc breach review by judging whether the trajectory correctly identified a real incident, missed one, or correctly closed a benign alert. Sparse but ground-truth-clean. Scale is binary (correct / incorrect) on the terminal decision, so a correctly-closed benign alert scores the same as a correctly-escalated incident, and a timeout (no decision.*) is excluded rather than scored

as incorrect.

LLM judge $r_j(t)$. Used to densify r_p in the steps where human labeling is too expensive at training scale. Calibrated against the human PRM set on a rolling 5% sample. If judge agreement against humans drops below threshold the judge is retrained, not used until it recovers. This is the antidote to the reward-hacking mode my DPBench paper surfaced, where an aggregate-good judge hides tail breakdowns.

Composition. Per-step reward uses the human process label where available, with the judge filling in elsewhere. Let $m_t \in \{0, 1\}$ indicate whether the human PRM label is present at step t :

$$r_t = \alpha \cdot m_t \cdot r_p(t) + \beta \cdot (1 - m_t) \cdot r_j(t)$$

Trajectory return averages per-step reward over the episode of length T and adds the terminal outcome (with $r_o = 0$ if the episode hits the step ceiling without a decision.*):

$$R = \frac{1}{T} \sum_{t=1}^T r_t + \gamma \cdot r_o$$

Averaging across the episode keeps the per-step term on the same order of magnitude as r_o regardless of episode length. A KL penalty against the reference (SFT) policy is applied directly in the GRPO loss (per the DeepSeekMath formulation, not folded into R), with its coefficient tuned to keep drift bounded.

Initial weights: $\alpha = 1.0$ (PRM dominant where labeled), $\beta = 0.5$ (judge as half-weight densifier so human-labeled checkpoints contribute twice the per-step weight of judge-only checkpoints), $\gamma = 1.0$ (terminal outcome same order as the averaged process reward). Note: γ here is a terminal-reward weight, not the standard RL discount factor. All pilot-tuned, not fixed.

For GRPO post-training, advantages are computed group-relatively within N trajectories sampled from the same starting alert. Let R_i denote the return of trajectory i in the group, and $\bar{R}_{\text{group}}, \sigma_{R_{\text{group}}}$ the mean and standard deviation across $\{R_1, \dots, R_N\}$:

$$A_i = \frac{R_i - \bar{R}_{\text{group}}}{\sigma_{R_{\text{group}}} + \epsilon}$$

where ϵ is a small constant for numerical stability when group reward variance is near zero. For rare-event outcome rewards I would also evaluate the unnormalized variant, since standard-deviation normalization is known to inflate advantages in low-variance groups (the Dr. GRPO critique). The same algorithmic structure as DPBench, with the reward sources changed.

Ablation plan. In the pilot I would sweep $\alpha \in \{0.5, 1.0, 1.5\}$, $\beta \in \{0, 0.25, 0.5, 0.75\}$, $\gamma \in \{0.5, 1.0, 2.0\}$, and the KL coefficient over half a decade each, and report process-reward and outcome-reward sensitivity on a held-out customer split. With episodes bounded at 30 steps and the open-weights baseline at 14B parameters, the full grid is cheap to evaluate; the point is to find the smallest configuration that does not collapse on either the rare-event outcome or the dense process signal.

Compute and curriculum. A single GRPO post-training run on a 14B base with 500 captured trajectories and group size $N = 8$ sits inside a one-person compute budget on rented H100s. The compound-alert curriculum runs in three stages: an SFT warm-up on Stage-3 bundles only; an early-RL phase with single-alert episodes and process-only rewards; and a late-RL phase that introduces compound alerts at 15% probability and turns on the terminal outcome signal. This keeps the policy from collapsing on the harder distribution before the dense process signal has shaped it, and confines the train/eval mismatch introduced by the compound mechanism to late-stage rollouts.

Quality control before declaring commercial feasibility

Test	Failure mode it catches	How it runs
Judge calibration	LLM judge gameable; reward hacking via prompt leakage	Sample 5% of episodes (min 100 per audit period), compare judge against human PRM, require quadratic-weighted $\kappa \geq 0.7$ on the 3-point ordinal
Inter-annotator κ	Process rubric is ambiguous between analysts	Two senior analysts independently score the same 200 checkpoints monthly; require quadratic-weighted $\kappa \geq 0.7$ on the 3-point ordinal scale
Leakage probe	Outcome predictable from alert metadata alone (task trivially solvable)	Train a gradient-boosted classifier on tabular alert fields plus a small encoder probe on free-text fields; require held-out AUC ≤ 0.55 with bootstrap 95% CI overlapping 0.5
Distribution shift	Policy overfits to one customer’s tooling or workflow	Train on customer A’s trajectories, evaluate on customer B’s; require mean process-reward drop $\leq 0.1\sigma$ of within-customer eval noise
Trajectory diversity	Captured episodes are too uniform	3-gram action-sequence entropy across episodes, conditioned on alert class; require within 80% of the human-trajectory entropy floor
Adversarial alerts	Brittleness to engineered edge cases	Synthetic FP and TP set (target 200 cases, balanced) built by a red-team analyst; process-reward drop ≤ 15 percentage points relative to clean eval
Playbook fidelity	PRM rubric does not match the firm’s actual playbook	Senior analyst reviews 50 episodes, signs off that rubric encodes the playbook
Adversarial reward audit	Reward function gameable by a Goodhart-style policy	Train a small policy whose only objective is reward (see Skalse et al. on reward hacking and Gao-Schulman-Hilton on reward over-optimization); reward-vs-outcome correlation under this policy should not exceed the random-policy baseline by more than 0.1
Sim-to-real parity	Mocked tool outputs diverge from real backends, breaking deployment	Replay 50 captured trajectories through both mocked and real backends; require action-distribution KL between the two below a small bound
Judge prompt injection	Free-text fields in case notes or analyst messages manipulate the LLM judge	Canary-string and jailbreak-corpus probes injected into analyst.message and case.update text; judge score on injected vs. clean must not differ by more than judge-noise variance
Eval ceiling	Frontier baseline already saturates the eval (task is dead on arrival)	Run Claude Opus 4.7, GPT-5.5, Gemini 3.1 Pro zero-shot on the 50-trajectory eval; require best frontier process-reward ≤ 0.75 on the workflow-faithful scoring (headroom for post-training to gain on)

Table 1: QC suite. Each row catches a specific failure mode of the task itself, not the policy. The task is declared commercially feasible only after all eleven thresholds clear before a pilot is presented to a lab buyer.

The adversarial reward audit is the test you would expect a serious vendor to run, and in practice the one most often skipped, arguably as important as judge calibration. A small policy that can grind out near-human reward through tool spam, redundant case updates, or judge-prompt manipulation is a sign the reward function is broken, not the policy. Catching this before training a 14B model on the task saves the compute and the credibility.

What this trains, and why a frontier lab would want it

Training on this task targets four capabilities current post-training pipelines undertrain:

- Tool selection under partial observability, where the right next tool depends on the last tool’s output rather than the prompt
- Long-horizon planning with rare-event terminal reward, the regime where dense process

reward is the only thing keeping the gradient signal alive

- Decision-making with incomplete information, where the agent must escalate or close without a definitive answer
- Adapting to mid-episode feedback from a human in the loop, which transfers directly to enterprise agent deployments

These transfer beyond SOC. Coding sessions over hours, research agents that synthesize evidence across sources, claims adjudication, and underwriting all share the same trajectory profile (long, multi-tool, multi-stakeholder, unverifiable at the trajectory level). A model post-trained on this task is the foundation for a family of vertical deployments, not just a SOC product.

What gets shipped

Two artifacts and one evaluation outcome.

The environment package (around 1000 lines of Python on Verifiers, with the eight tools, the typed action space, the reward composition, the rollout runner, and the QC test suite). A 50-trajectory eval set carved out of the 500 trajectories captured in the neglected-work pilot, scored on process-reward agreement with senior analysts and outcome-reward agreement on the rare-event tail. A baseline rollout report comparing Claude Opus 4.7, GPT-5.5, Gemini 3.1 Pro, and open-weights baselines (Qwen3-14B post-trained with the pipeline, GLM 5.1 for reference) on this eval.

The eval report is the artifact a frontier lab would actually read. If a post-trained 14B beats frontier closed-source models on this workflow-faithful eval by ten points on process reward, the conversation with Anthropic moves from "buy data" to "co-develop a capability." That is the conversation worth having.